

GrantMatch Build Runbook

Reference for building the MVP. Two sibling labs already contain most of the pieces — adapt, don't rewrite from zero.

Source map

Component	Pull from	File(s)	Action
PDF/doc loader	rag-chunking-lab	shared/loader.py	reuse as-is
Chunking	rag-chunking-lab	chunking/section_wise.py	reuse as-is — rule docs have headers (Eligibility, Income Limits, Deadlines)
Vector store	rag-chunking-lab	vectordb/chroma_store.py	reuse as-is, near copy-paste
Embedding	rag-chunking-lab (pattern)	shared/embedder.py	REWRITE — swap sentence-transformers for OpenAI text-embedding-3-small
Baseline LLM call	rag-chunking-lab (pattern)	shared/llm.py, rag/pipeline.py	REWRITE — swap Anthropic for OpenAI; keep system/user split
Hybrid retrieval	rag-chunking-lab	retrieval/hybrid.py, bm25_search.py	defer past MVP — plain ChromaStore.search() is enough for v1
Agent loop	single-agent-lab	agent.py	reuse structure — Agent(), tool_plain, output_type=[A,B], UsageLimits
Typed output shape	single-agent-lab	schemas.py	reuse pattern, new fields
Web search tool	single-agent-lab	tools/web_search.py	reuse as-is (DuckDuckGo, keyless)
Tracing	single-agent-lab	trace.py	reuse as-is, only tool names change
UI pattern	single-agent-lab	app.py	reference only — design doc calls for Streamlit, not Gradio
Eval harness	rag-chunking-lab	eval/metrics.py	adapt — swap recall@k for eligibility/citation accuracy

Concepts — what each piece actually teaches

This is the part worth reading slowly. Everything below is a real mechanism used in the code you're about to adapt, not a generic explainer — each one names the file it lives in.

Ingestion & retrieval (phase 1)

Chunking strategies — chunking/section_wise.py, chunking/recursive.py. A chunker breaks a document into pieces small enough to embed and retrieve individually. Recursive splitting tries coarse separators first (paragraph → line → space → character) and backs off to finer ones only when a piece is still too big — it guarantees a size limit but knows nothing about meaning. Section-wise chunking instead finds structural headers (regex-matched: “Abstract,” “Eligibility,” “Income Limits”) and keeps each section whole, only re-splitting if a section runs long. GrantMatch uses section-wise because a rule doc split mid-clause loses the exact fact you'd need to cite — this is RAG_LESSONS.md lesson #1, discovered when a paper's Abstract got cut in half and neither half made sense alone.

Embeddings — shared/embedder.py. An embedding turns text into a vector positioned so that semantically similar text ends up close together, measured by cosine similarity. Model size matters: rag-chunking-lab found that switching from a 384-dimension model to a 768-dimension one fixed a real failure — the query “LLM as a judge” wasn't matching a paper's actual phrasing (“prompting an LLM to...”) at 384d, but did at 768d. GrantMatch uses OpenAI's text-embedding-3-small instead of a local sentence-transformers model — same concept, hosted API instead of local inference.

Vector search / HNSW — vectordb/chroma_store.py. Once every chunk is a vector, retrieval means finding the k vectors closest to the query's vector. ChromaDB does this with HNSW (Hierarchical Navigable Small World) — a multi-

layer graph where each vector links to its nearest neighbors, giving roughly $O(\log n)$ approximate search instead of comparing against every vector one by one. At GrantMatch's scale (a few hundred chunks from 10 programs) brute-force search would be just as fast — HNSW only starts winning past ~100K vectors — but the API is identical either way.

Hybrid retrieval & query rewriting — `retrieval/hybrid.py`, `query_rewriter.py` (deferred). Dense (embedding) search understands meaning but can miss exact keywords; BM25 sparse search is the reverse. Hybrid search runs both and merges the ranked lists with Reciprocal Rank Fusion. Query rewriting (HyDE, RAG-Fusion) goes further and has an LLM reformulate the query before embedding it, to bridge the gap between how a student phrases a question and how a rule document phrases the same fact. Worth knowing, not needed to prove the MVP works.

Prompt injection defense — `shared/llm.py` pattern. If a retrieved chunk contains something that reads like an instruction, an LLM can follow it instead of answering the user — this happened for real in `rag-chunking-lab` (`RAG_LESSONS.md` #4), where a paper's own quoted methodology text made the model respond “Insufficient Information” to a normal question. The fix is structural: instructions go in the system message, retrieved content goes in the user message, with explicit framing that it's source material, not instructions to obey.

Agent loop (phase 2)

ReAct loop — `single-agent-lab/agent.py`, `trace.py`. “agent = LLM + tools + loop.” Pydantic AI's Agent runs a Thought → Action → Observation cycle: the model reasons about what it needs, calls a tool, reads the real result, and either loops again or produces a final answer. Same shape whether the domain is travel briefings or scholarship eligibility — only the tools and system prompt change.

Typed outputs & validation — `schemas.py`. Every tool's arguments and the agent's final answer are Pydantic models, so JSON Schema alone rejects malformed output before your code runs — that's schema validation. Business-logic validation (syntactically valid but wrong, like a briefing with no destination) is caught separately with `@agent.output_validator`, which raises `ModelRetry` to send the model back around the loop with the specific reason. GrantMatch's version: `output_type=[EligibilityAnswer, NeedMoreInfo]`, mirroring `single-agent-lab`'s `TravelBriefing/NeedMoreInfo` union — succeed with a grounded answer, or admit you can't.

Tool design & dependent chains — `tools/geocode.py` → `get_forecast` pattern. The clean example in `single-agent-lab`: `get_forecast` needs coordinates, and the model cannot invent them from memory, so it must call `geocode` first and feed that real result forward. That's “step N's input depends on step N-1's output,” enforced by types, not convention. GrantMatch's `retriever_tool` plays the same role — the agent cannot cite a clause it didn't actually retrieve.

Guardrails — `UsageLimits`, per-tool retries. Two separate budgets bound the loop: `UsageLimits` caps total requests and tool calls for an entire run (stops a runaway request from spiraling into 40 calls), while each `tool_plain(retries=N)` gives that one tool its own retry budget when it raises `ModelRetry` or gets invalid arguments. Different failure, different counter — an agent loop without a cap can silently burn API budget.

Tracing / observability — `trace.py`. Pydantic AI hands back the full message history via `result.all_messages()`; `trace.py` just walks it and labels each part (ThinkingPart → Thought, ToolCallPart → Action, ToolReturnPart → Observation) so the loop is visible instead of a black box. This is what lets you debug “why did the agent do that” instead of guessing.

Evaluation (phase 3)

Eval metric design — `eval/metrics.py`, `RAG_LESSONS.md` #7. `recall@k` and MRR (mean reciprocal rank) are the standard retrieval metrics — did the right chunk come back, and how high did it rank. `rag-chunking-lab`'s version uses substring keyword matching, which is fast but fragile (a word like “attention” matches a paper title in the References section that has nothing to do with the real answer). GrantMatch's equivalent metrics — eligibility accuracy, citation accuracy, hallucination rate — carry the same fragility risk: citation accuracy has to check the cited clause actually matches the source, not just that some keyword appears in it.

Lessons to apply directly

From `rag-chunking-lab/RAG_LESSONS.md`:

- **Chunk on structure, not characters.** A scholarship's "Income Limit" clause split in half loses its meaning — this is why `section_wise.py`, not `recursive.py`, is the default chunker here.
- **System/user message separation prevents prompt injection.** If a rule doc contains sample application text or quoted instructions, the model can confuse it with a real instruction. Keep context and instructions in separate message roles, with explicit framing.
- **Vector DB choice doesn't matter at this scale.** Don't spend MVP time comparing stores — Chroma is fine.
- **Don't cheap out on the generation model.** A weak model gives up on dense, clause-heavy text. Use a solid OpenAI model for the eligibility reasoning step.
- **Keyword-substring eval metrics are fragile.** For citation accuracy, check the cited clause actually matches the source text — don't just check a keyword is present.
- **Know your embedding model's context limit** and make sure whole clauses aren't silently truncated during embedding.

New pieces (no existing code to pull from)

`schemas.py` — mirrors single-agent-lab's `TravelBriefing` / `NeedMoreInfo` union:

```
class EligibilityAnswer(BaseModel):
    program: str
    eligible: Literal["yes", "no", "partial"]
    cited_clause: str      # the exact rule text
    cited_source: str      # program name / doc it came from
    reasoning: str
    caveats: list[str] = []

class NeedMoreInfo(BaseModel):
    question: str
    reason: str
```

`retriever_tool.py` — thin wrapper: `embed_query(text) → ChromaStore.search(embedding, k) → list of {text, metadata, score}`, registered with `agent.tool_plain`.

System prompt — must state: report eligibility only, not odds of winning; always cite the exact clause; flag partial matches instead of forcing yes/no; use the retriever tool first, web search only if the program isn't in the local corpus.

Data — 10-15 scholarship/grant rule docs + 25-30 hand-labeled student test profiles with ground truth.

Proposed repo layout

```
Capstone-Project/
  ingest.py          # load -> chunk -> embed -> ChromaDB.add
  chunking/section_wise.py # copied from rag-chunking-lab
  vectordb/chroma_store.py # copied from rag-chunking-lab
  shared/embedder.py   # rewritten for OpenAI embeddings
  agent/
    schemas.py        # EligibilityAnswer / NeedMoreInfo
    tools.py          # retriever_tool + web_search (copied)
    agent.py          # adapted from single-agent-lab/agent.py
    trace.py          # copied, tool names updated
  eval/
    golden_profiles.json # 25-30 labeled student profiles
    run_eval.py         # eligibility / citation / hallucination scorers
  data/programs/      # source rule PDFs (loader.py reads *.pdf)
  app.py              # Streamlit UI
  requirements.txt
  .env
```

Build order

Phase 1 — ingestion + baseline

- Pick 10-15 programs with clear, checkable rules; collect the docs into data/programs/ — **done**
- Copy section_wise.py and chroma_store.py verbatim *concept: chunking strategies, vector search/HNSW*
- Write shared/embedder.py for OpenAI text-embedding-3-small *concept: embeddings*
- Write ingest.py, run it once
- Smoke-test: query Chroma directly on 3-4 sample questions, confirm the right chunks come back

Phase 2 — agent

- Write agent/schemas.py *concept: typed outputs & validation*
- Write agent/tools.py — retriever tool + copy web_search.py *concept: tool design & dependent chains*
- Adapt agent.py: new system prompt, output_type=[EligibilityAnswer, NeedMoreInfo], register both tools, keep UsageLimits *concept: ReAct loop, guardrails*
- Copy trace.py, run 5-10 manual queries, eyeball the traces *concept: tracing/observability*

Phase 3 — eval, UI, wrap-up

- Build golden_profiles.json (25-30 labeled profiles)
- Adapt eval/metrics.py into eligibility/citation/hallucination scorers *concept: eval metric design*
- Run baseline (vanilla RAG) vs agent on the same test set, log the comparison
- Build the Streamlit UI
- Write the comparative-eval conclusion + failure log for the README
- Record the 3-minute demo

Data collected

10 real programs, each as a one-page PDF rule sheet in data/programs/ — converted from plain text to PDF specifically so rag-chunking-lab's shared/loader.py (PdfReader-based, globs *.pdf) can load them without modification. Verified each one round-trips cleanly through pypdf.PdfReader(...).extract_text(). Sourced from official program pages (state higher-ed agencies + studentaid.gov). Mix of rule types on purpose — flat income cutoffs, income formulas, GPA-only, GPA + test score + service hours, and categorical (residency, first-gen, degree status) — so the eval set can exercise different citation patterns:

File	Program	Rule type
nj_tag.pdf	NJ Tuition Aid Grant	income + residency + degree status
nj_stars.pdf	NJ STARS / STARS II	GPA + class rank + income
federal_pell_grant.pdf	Federal Pell Grant	SAI formula (not flat income)
georgia_hope_scholarship.pdf	Georgia HOPE Scholarship	GPA + residency
california_cal_grant.pdf	California Cal Grant A/B	GPA + income/asset ceiling
florida_bright_futures.pdf	Florida Bright Futures (FAS)	GPA + SAT + service hours
federal_teach_grant.pdf	Federal TEACH Grant	GPA or test percentile + service commitment
nj_eof.pdf	NJ Educational Opportunity Fund	income + disadvantaged background + first-gen
federal_seog.pdf	Federal SEOG	need-based, funding-limited
ny_tap.pdf	NY Tuition Assistance Program	income tiers by dependency status

Numbers are as reported by official sources at research time (mid-2026) — re-verify against the live source URL in each file before treating any figure as final, since aid thresholds change year to year.

Open decisions before starting

- OpenAI embeddings (design doc default) vs reusing sentence-transformers to save API cost — confirm key access first (note: .env already has OPENAI_API_KEY set, so this looks resolved)